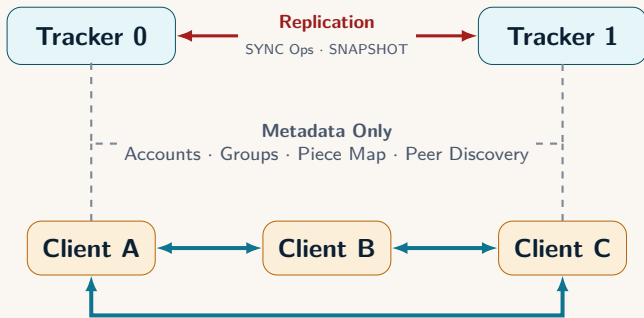


Peer-to-Peer Distributed File Sharing System

Group-Based Swarm Downloads Over a Fault-Tolerant Control Plane



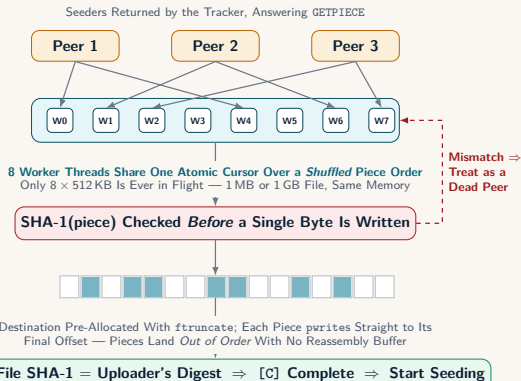
The System in One Line

Users authenticate, form groups, and share files. The tracker pair stores only **metadata**; a download is assembled from **512 KB pieces pulled from every available seeder in parallel** and verified by **SHA-1** at both the piece and whole-file level.

512 KB Piece Transfers — File Data Never Touches a Tracker

Every Client Is Simultaneously Seeder and Leecher

Download Engine: Parallel Swarm Transfer With Verified Pieces



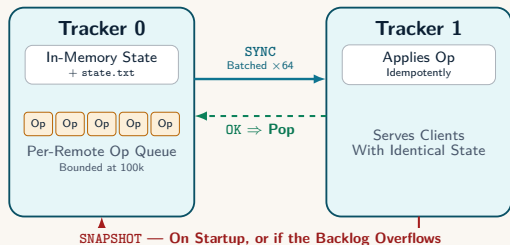
Getting Parallelism Right

- ▶ **No duplicated work:** workers claim pieces from a shared atomic cursor, so eight threads never race for the same index.
- ▶ **No hot-spotting:** each worker enters the peer list at a different offset, $(\text{worker} + \text{slot} + k) \% \text{npeers}$ — requests fan out across the swarm instead of stampeding peer 0.
- ▶ **Randomised piece order** spreads *different* leechers of one file onto different seeders. Two attempts per peer, then fall through.

Integrity Is Not a Post-Check

A corrupt piece is **indistinguishable from a failed peer** — discarded before the write, then refetched elsewhere. The whole-file hash is a second, independent gate.

Staying Up: Tracker Replication and Transparent Client Failover



Peer Unreachable ⇒ Backoff 1s → 2s → 4s → ... → 30s, Nothing Dropped

Why a Downed Tracker Loses Nothing

- ▶ Ops stay queued **until acknowledged** and replay *in order* when the peer returns; a worker thread per remote keeps replication off the request path.
- ▶ An overflowing backlog falls back to a full **SNAPSHOT** — the same path that recovers a long-offline tracker.

Clients Fail Over Too

A failed request is retried down the tracker list, and `ERR not_logged_in` triggers a **transparent re-login and replay** — a tracker can be killed mid-session without interrupting the user.

Known Limits — Stated, Not Hidden

Plain-text credentials, client-asserted identity — a trusted-network design; whole-state rewrite per change.